

Chapter 7

Python

String Operations

Presented by:
Amit Roy
MSc Student at IIT

7.1 What is a String?

The building block of text in Python

Definition

A String is simply a piece of text — words, sentences, or any collection of characters.

Immutable

Once created, you can't change its original content. Python creates a new string when you modify.

3 Ways to Create

Single quotes `' '`, double quotes `" "`, or triple quotes `"""` for multi-line text.



```
s1 = 'Hello'
s2 = "World"
s3 = '''This is
a multi-line
string'''
```

```
print(type(s1))           # <class 'str'>
print(s1 + " " + s2)      # Hello World
print(s3)                 # Prints as written
```

7.1.1 & 7.1.2 Length & Whitespace

Measuring and cleaning strings

String Length — len()

The len() function counts every character — letters, spaces, and symbols.



```
text = "Hello World"
print(len(text))
# 11 (H-e-l-l-o-space-W-o-r-l-d = 11 characters)
```

Whitespace Removal — strip()

Removes invisible characters (spaces, tabs \t, newlines \n) from start & end.



```
sentence = "\n\n\n Hello World \t" # \n = new line, \t = tab
print(sentence.strip())           # 'Hello World' - Cleaned up
```

Visual: Each character has an index position

H	e	l	l	o		W	o	r	l	d
0	1	2	3	4	5	6	7	8	9	10

Index →

How strip() works

Before: " Python "

strip(): "Python"

lstrip(): "Python "

rstrip(): " Python"

7.1.3 Replace

String Replacement — .replace()

Finds ALL occurrences of a word and replaces them with another.

```
● ● ●
text = "Hello World"
new_text = text.replace("World", "Python")    # Find "World" and replace with "Python"
print(new_text)    # Hello Python
print(text)        # Still "Hello World"
```

replace() — Before & After

Before: "Hello World"

After: "Hello Python"

7.1.4 Formatting

✦ f-Strings (String Formatting)

Put an f before quotes, then use {} to insert variables — neat and readable!

```
name = "Alice"  
age = 25  
print(f"My name is {name} and I am {age} years old.")
```

f-String Magic ✦

Output:

```
My name is Alice and I am 25 years old.
```

7.1.5 Concatenation

+ Concatenation — Joining Strings



```
first_name = "John"
last_name = "Doe"
full_name = first_name + " " + last_name
print(full_name)
```



How Concatenation Works

"John" + " " + "Doe"



"John Doe"



```
s = "Age: "
n = 30
print(s + n)
```

This would cause an error because you can't directly join text and numbers

TypeError: can only concatenate str (not "int") to str



```
s = "Age: "
n = 30
print(s + str(n))
```

Age: 30 - n is first converted into string, 30 becomes '30',
then it is concatenated.

7.1.6 Repetition

✕ Repetition — Repeat with *



```
greet = "Hi"  
print(greet * 3)
```

Output

HiHiHi



```
separator = "-" * 20  
print(separator)
```

Output

7.1.7 & 7.1.9 Indexing & Slicing



Slicing — `text[start:end]`

Start is included, end is NOT ("up to but not including")

`text[0:3] = ?`



Result: "Pyt"

7.1.7 & 7.1.9 Indexing & Slicing



```
text = "Python"

print(text[0])    # P (first character)
print(text[3])    # h (fourth character)
print(text[-1])   # n (last character)
print(text[-3])   # h (third from the end)
```

Output

```
# P          (first character)
# h          (fourth character)
# n          (last character)
# h          (third from the end)
```



```
text = "Python"

print(text[0:3])
print(text[2:])
print(text[:4])
print(text[-4:-1])
```

Output

```
# Pyt (positions 0,1,2 - stops before position 3)
# thon (from position 2 to the end)
# Pyth (from the beginning up to position 4)
# tho (using negative indices)
```

7.1.8 Mutability of Strings



```
s = "Hello"  
s[0] = "J"
```

```
# This will raise - TypeError: 'str'  
object does not support item assignment
```



```
# CORRECT WAY
```

```
s = "Hello"  
s = "J" + s[1:]  
print(s)
```

Output

Jello

Split



```
sentence = "Python is easy to learn"  
words = sentence.split()  
print(words)
```

```
# Split at spaces (default behavior)  
Output:  
['Python', 'is', 'easy', 'to', 'learn']
```



You can specify what to split on

```
csv = "apple,banana,cherry"  
fruits = csv.split(",")  
print(fruits)
```

```
# Split at each comma  
Output:  
['apple', 'banana', 'cherry']
```

Substring find()



```
text = "Hello World"

print(text.find("World"))
print(text.find("Python"))
```

```
# 6 (the word "World" starts at position 6)
# -1 (not found - Python returns -1 when searching fails)
```

```
# This is useful for checking if something exists before trying to work with it.
# Confused seeing 'if'? Don't worry, we will learn it clearly in upcoming lectures
```



```
if text.find("World") != -1:
    print("Found 'World' in the text!")
```

Occurance



```
text = "banana"
```

```
print(text.count("a"))  
print(text.count("na"))
```

```
# 3 (the letter 'a' appears 3 times)  
# 2 (the sequence 'na' appears 2 times)
```

```
# This is useful for analysis or validation
```



```
password = "password123"  
if password.count("1") > 0:  
    print("Password contains at least one '1'")
```

Case_Changing



```
message = "Python is FUN"
```

```
print(message.lower())
```

```
print(message.upper())
```

```
# You can make first character of every word capital
```

```
print(message.title())
```

```
# Output: python is fun
```

```
# Output: PYTHON IS FUN
```

```
# Output: Python Is Fun
```

Check start_with , end_with



```
filename = "report2025.csv"

print(filename.startswith("report"))
print(filename.endswith(".csv"))
print(filename.endswith(".txt"))
```

```
# Output: True
# Output: True
# Output: False
```

7.1.11 – 7.1.14 Useful String Methods

Finding, counting, changing case & more

`.find(sub)`

```
text.find("World") # 6
```

Returns position, -1 if not found

`.count(sub)`

```
"banana".count("a") # 3
```

Counts occurrences

`.lower()`

```
"HELLO".lower() # hello
```

All characters lowercase

`.upper()`

```
"hello".upper() # HELLO
```

All characters uppercase

`.title()`

```
"hello world".title() # Hello World
```

First letter of each word capitalized

`.split(sep)`

```
"a,b,c".split(",") # ['a','b','c']
```

Breaks into a list

`.startswith()`

```
"report.csv".startswith("report")
```

Returns True/False

`.endswith()`

```
"file.csv".endswith(".csv")
```

Returns True/False

Practice Exercises

Try these yourself!

1. Change Case Practice

Take a user input string. Print it in lowercase, UPPERCASE, and Title Case.

2. Filename Filter

From a list, print only filenames that start with "data" and end with ".csv".

3. Email Domain Checker

Check if an email ends with "@gmail.com". Print "Gmail account" or "Other provider".

4. Custom Hashtag Generator

Input: " Python Is Awesome " → Output: #python_is_awesome
(strip, lower, replace spaces with _)

5. Validate Password Start & End

Check if password starts with a letter and ends with a digit (0-9). Print Valid/Invalid.

Sample Solutions

One possible answer for each exercise



Exercise 1: Change Case

```
text = input("Enter text: ")
print(text.lower())
print(text.upper())
print(text.title())
```



Exercise 2: Filename Filter

```
files =
["data1.csv", "image.png", "data_report.csv", "summary.txt"]
for f in files:
    if f.startswith("data") and f.endswith(".csv"):
        print(f)
```



Exercise 3: Email Domain Checker

```
email = input("Enter email: ")
if email.endswith("@gmail.com"):
    print("Gmail account detected")
else:
    print("Other email provider")
```



Exercise 4: Hashtag Generator

```
sentence = input("Enter sentence: ")
hashtag = "#" + sentence.strip().lower().replace(" ",
"_")
print(hashtag)
```



Exercise 5: Validate Password

```
password = input("Enter password: ")
letters = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
digits = "0123456789"
special = "!@#$%^&*()-_+=~`~'~\"
```

Chapter 7 — Quick Summary

 **String:** A sequence of characters. Created with ' ', " ", or "" "".

 **len():** Returns total number of characters including spaces.

 **replace():** Replaces all occurrences of a substring with another.

 **Indexing:** Access characters by position: text[0], text[-1].

 **find()/count():** Locate or count substrings.

 **split():** Break string into a list using a separator.

 **Immutable:** You can't change a character in-place. Build a new string.

 **strip():** Removes leading/trailing whitespace from a string.

 **f-strings:** Insert variables directly: f"Hello {name}"

 **Slicing:** Extract portion: text[start:end] (end not included).

 **Case methods:** .lower(), .upper(), .title() to change case.

 **startswith/endswith:** Check how a string begins or ends.